

CS345

Assignment 1

Chaitanya
Roll Number: 230316

Due Date: 31st August

Model of computation: unless stated otherwise, we work in the standard unit-cost RAM model, in which one arithmetic operation (addition, multiplication, comparison) on values that fit in a machine word takes $O(1)$ time. This is the model implicitly assumed by the course (“Assume that sorting would have $O(n \log n)$ complexity”).

Contents

1	Problem 1: The n-th Root of m via Divide and Conquer (10 points)	2
1.1	Algorithm Idea & Intuition	2
1.2	Formal Pseudocode	3
1.3	Complexity Analysis	3
1.4	Proof of Correctness	4
2	Problem 2: Point-in-Polygon Test in $O(n)$ Time (10 points)	6
2.1	Algorithm Idea & Intuition	6
2.2	Formal Pseudocode	7
2.3	Complexity Analysis	7
2.4	Proof of Correctness	8
3	Problem 3: Minimizing Weighted Sum of Completion Times (10 points)	10
3.1	Algorithm Idea & Intuition	10
3.2	Formal Pseudocode	10
3.3	Complexity Analysis	10
3.4	Proof of Correctness (Exchange Argument)	11
4	Problem 4: Cheapest Reachable Node in a DAG (10 points)	13
4.1	Algorithm Idea & Intuition	13
4.2	Formal Pseudocode	14
4.3	Complexity Analysis	14
4.4	Proof of Correctness	15

1 Problem 1: The n -th Root of m via Divide and Conquer (10 points)

Problem Statement

Given two positive integers n and m , design an $O(n \log m)$ -time Divide-and-Conquer algorithm to compute $\sqrt[n]{m}$, i.e., to find the natural number x such that $x^n = m$. It is guaranteed that such a natural number x exists.

1.1 Algorithm Idea & Intuition

The key structural observation is *monotonicity*: for a fixed exponent $n \geq 1$, the function $f(x) = x^n$ is strictly increasing in x over the positive integers. This is exactly the structural property that makes **binary search** applicable: we can search for the value x^* satisfying $(x^*)^n = m$ by repeatedly halving a candidate interval $[lo, hi]$ that is guaranteed to contain x^* , comparing mid^n against m at each step, and discarding the half of the interval that cannot contain x^* .

Two ingredients are combined:

1. **Divide and Conquer on the search space (Binary Search):** Maintain an interval $[lo, hi]$ known to contain x^* . At each step, probe the midpoint mid , and recurse into only one half — this is a "decrease-and-conquer" instance of the D&C paradigm, halving the problem size $b = 2$ with $a = 1$ subproblem solved per level.
2. **Fast (Binary) Exponentiation:** To evaluate the probe mid^n efficiently (rather than via $n-1$ naive multiplications), we use repeated squaring: writing n in binary, x^n can be computed using only $O(\log n)$ multiplications via the recurrence

$$x^n = \begin{cases} 1 & n = 0 \\ (x^{\lfloor n/2 \rfloor})^2 & n > 0, n \text{ even} \\ (x^{\lfloor n/2 \rfloor})^2 \cdot x & n > 0, n \text{ odd.} \end{cases}$$

This itself is a Divide-and-Conquer algorithm on the exponent.

Since $x^* \geq 1$ and $n \geq 1$ imply $(x^*)^n \geq x^*$ (raising a number ≥ 1 to a power ≥ 1 never decreases it), and $(x^*)^n = m$, we get $x^* \leq m$. Hence the initial search interval $[1, m]$ is safe (guaranteed to contain x^*).

1.2 Formal Pseudocode

Algorithm 1: Divide-and-Conquer Integer n -th Root via Fast Exponentiation

Input: Positive integers n (degree) and m (radicand); it is guaranteed that a natural number x with $x^n = m$ exists

Output: The unique natural number x such that $x^n = m$

```

1 Function FastPow( $x, n$ ):
2   if  $n = 0$  then
3     return 1;
4    $h \leftarrow \text{FastPow}(x, \lfloor n/2 \rfloor)$ ;
5   if  $n \bmod 2 = 0$  then
6     return  $h \cdot h$ ;
7   else
8     return  $h \cdot h \cdot x$ ;

9 Function NthRoot( $n, m, lo, hi$ ):
   // Invariant:  $x^*$  satisfying  $(x^*)^n = m$  lies in  $[lo, hi]$ 
10  if  $lo > hi$  then
11    return FAIL;           // unreachable under the problem's guarantee
12   $mid \leftarrow \lfloor (lo + hi)/2 \rfloor$ ;
13   $v \leftarrow \text{FastPow}(mid, n)$ ;
14  if  $v = m$  then
15    return  $mid$ ;
16  else if  $v < m$  then
17    return NthRoot( $n, m, mid + 1, hi$ );
18  else
19    return NthRoot( $n, m, lo, mid - 1$ );

20 Main call: return NthRoot( $n, m, 1, m$ );

```

1.3 Complexity Analysis

Claimed bounds. Time: $O(\log n \log m)$ (which is a subset of, and hence certainly satisfies, the required $O(n \log m)$ bound, since $\log n = O(n)$). Space: $O(\log n + \log m)$ (recursion stack only).

Cost of FastPow. Let $M(n)$ denote the number of multiplications performed by **FastPow**(x, n). Then

$$M(n) = M(\lfloor n/2 \rfloor) + O(1), \quad M(0) = 0,$$

since each call does at most one recursive call plus $O(1)$ multiplications ($h \cdot h$, and possibly $h \cdot h \cdot x$). By the Master theorem (or direct unrolling: the recursion depth is $\lfloor \log_2 n \rfloor + 1$, each level doing $O(1)$ work), we get

$$M(n) = O(\log n).$$

Hence a single call to **FastPow**() runs in $O(\log n)$ time.

Cost of NthRoot. Let $T(k)$ be the running time of **NthRoot**() on an interval of size $k = hi - lo + 1$. Each call performs $O(1)$ arithmetic work plus one call to **FastPow**() (costing $O(\log n)$),

and recurses into a sub-interval of size at most $\lceil k/2 \rceil$. This gives the recurrence

$$T(k) = T(\lceil k/2 \rceil) + O(\log n), \quad T(1) = O(\log n).$$

This is precisely the recurrence solved by the Master theorem with $a = 1, b = 2, f(k) = O(\log n)$ (a constant with respect to k). Since $f(k) = \Theta(k^{\log_b a}) = \Theta(k^0) = \Theta(1)$ up to the $\log n$ factor, Case 2 of the Master theorem gives

$$T(k) = O(f(k) \cdot \log k) = O(\log n \cdot \log k).$$

With the initial interval size $k = m$ (the range $[1, m]$), we obtain

$$T(m) = O(\log n \log m).$$

Because $\log n = O(n)$, this is $O(n \log m)$, so the algorithm meets (and in fact asymptotically beats) the target bound.

Exact step count. The number of recursive calls made by `NthRoot()` is at most $\lfloor \log_2 m \rfloor + 1$ (standard binary-search depth bound on a universe of size m), and each call invokes `FastPow()` which performs at most $2\lfloor \log_2 n \rfloor$ multiplications. Hence the total number of multiplications performed by the whole algorithm is bounded by

$$(\lfloor \log_2 m \rfloor + 1) \cdot 2\lfloor \log_2 n \rfloor = O(\log n \log m).$$

Space. The recursion depth of `NthRoot()` is $O(\log m)$ and, within each call, `FastPow()` adds a further $O(\log n)$ recursion depth; each stack frame uses $O(1)$ space. Hence total space is $O(\log n + \log m)$.

1.4 Proof of Correctness

Lemma 1 (Strict Monotonicity). *For any fixed integer $n \geq 1$, the function $f(x) = x^n$ is strictly increasing for $x \geq 1$: if $1 \leq x_1 < x_2$ then $x_1^n < x_2^n$.*

Proof. By induction on n . *Base case* $n = 1$: $f(x) = x$ is trivially strictly increasing. *Inductive step*: assume $x_1^{n-1} < x_2^{n-1}$ for $1 \leq x_1 < x_2$. Multiplying both sides of $x_1^{n-1} < x_2^{n-1}$ by the positive number x_1 gives $x_1^n < x_1 x_2^{n-1}$, and multiplying $x_1 < x_2$ by the positive number x_2^{n-1} gives $x_1 x_2^{n-1} < x_2^n$. Chaining, $x_1^n < x_2^n$. ■ □

Lemma 2 (Correctness of FASTPOW). *For all $x \geq 0$ and integers $n \geq 0$, `FastPow`(x, n) returns x^n .*

Proof. Strong induction on n . *Base case* $n = 0$: the function returns $1 = x^0$, correct. *Inductive step*: let $n \geq 1$ and assume the claim holds for every $n' < n$; in particular for $n' = \lfloor n/2 \rfloor < n$. By the induction hypothesis, $h = x^{\lfloor n/2 \rfloor}$.

- If n is even, $n = 2\lfloor n/2 \rfloor$, so $x^n = (x^{\lfloor n/2 \rfloor})^2 = h \cdot h$, which is exactly what is returned.
- If n is odd, $n = 2\lfloor n/2 \rfloor + 1$, so $x^n = (x^{\lfloor n/2 \rfloor})^2 \cdot x = h \cdot h \cdot x$, which is exactly what is returned.

■ □

Invariant 1 (Search Interval Invariant for `NthRoot()`). At every invocation `NthRoot( $n, m, lo, hi$ )` made by the algorithm (starting from the initial call `NthRoot( $n, m, 1, m$ )`), the unique natural number x^* with $(x^*)^n = m$ satisfies $x^* \in [lo, hi]$, *provided* x^* still lies within the original range (which it does, by the argument below).

Theorem 1 (Correctness of Algorithm 1). *The algorithm returns x^* , the unique natural number with $(x^*)^n = m$.*

Proof. Base of the invariant. Since $x^* \geq 1$ and $n \geq 1$, and since $y \mapsto y^n$ satisfies $y^n \geq y$ for $y \geq 1$ (immediate from Lemma 1 applied to $1 \leq y$, or directly by induction on n), we have $m = (x^*)^n \geq x^*$. Hence $x^* \in [1, m]$, establishing the invariant for the initial call.

Inductive preservation. Suppose the invariant holds at a call `NthRoot( $n, m, lo, hi$ )`, i.e., $x^* \in [lo, hi]$, and suppose $lo \leq hi$ (otherwise, by the invariant, $[lo, hi]$ is empty only if x^* has already been found and returned in a prior call — we verify this cannot terminate the invariant improperly, see the termination case below). Let $mid = \lfloor (lo + hi)/2 \rfloor$ and, by Lemma 2, $v = mid^n$ exactly.

- If $v = m$, then $mid^n = m = (x^*)^n$; since $x \mapsto x^n$ is injective on the naturals by Lemma 1 (strict monotonicity implies injectivity), $mid = x^*$, and the algorithm correctly returns x^* .
- If $v < m$, i.e., $mid^n < (x^*)^n$, Lemma 1 (contrapositive: if $x^* \leq mid$ then $(x^*)^n \leq mid^n$) forces $x^* > mid$, i.e., $x^* \in [mid + 1, hi]$. The algorithm recurses on exactly $[mid + 1, hi]$, preserving the invariant.
- If $v > m$, symmetrically $x^* < mid$, i.e., $x^* \in [lo, mid - 1]$, and the algorithm recurses on exactly this interval, preserving the invariant.

Termination and correctness of output. Each recursive call strictly decreases $hi - lo$ (the interval size is at least halved), so after at most $\lfloor \log_2 m \rfloor + 1$ calls the algorithm either returns via the $v = m$ branch, or reaches $lo > hi$. Since the invariant guarantees $x^* \in [lo, hi]$ whenever $lo \leq hi$, and x^* is a fixed natural number, the interval can only become empty *after* x^* has already been found and returned (it cannot “fall out” of a shrinking interval that always contains it while $lo \leq hi$). Hence the $lo > hi$ branch is unreachable under the problem’s guarantee that x^* exists, and the algorithm always terminates by returning $mid = x^*$ in the equality branch. ■ □

2 Problem 2: Point-in-Polygon Test in $O(n)$ Time (10 points)

Problem Statement

Given a (possibly non-convex) simple polygon P on n points and a query point α , design an $O(n)$ -time algorithm to determine whether α lies inside P .

2.1 Algorithm Idea & Intuition

We use the classical **Ray Casting (Even–Odd Rule)** technique. Fix the polygon as an ordered list of vertices $v_0, v_1, \dots, v_{n-1}$ (indices mod n), with edges $e_i = (v_i, v_{i+1 \bmod n})$. The intuition, grounded in the *Jordan Curve Theorem* for simple closed curves, is:

Shoot a ray from α to infinity in a fixed direction (we use the $+x$ direction). Every time the ray crosses the polygon boundary, it toggles between being “inside” and “outside” the region enclosed by the boundary. Far away along the ray, one is certainly outside the (bounded) polygon. Hence α is inside the polygon if and only if the ray crosses the boundary an *odd* number of times.

Implementing this naively is subtle because of three degeneracies that must be handled with care:

- (a) **The ray passes exactly through a vertex** of the polygon (a shared endpoint of two consecutive edges has the same y -coordinate as α).
- (b) **An edge is horizontal** (both endpoints share α ’s y -coordinate, or lie collinear with the ray).
- (c) α **itself lies exactly on the boundary** of P (neither strictly inside nor outside).

We resolve (a) and (b) using the standard asymmetric strict-inequality convention (used in W. Randolph Franklin’s PNPOLY test): an edge (v_i, v_j) is counted as crossing the horizontal ray through α if and only if

$$(v_i.y > \alpha.y) \neq (v_j.y > \alpha.y),$$

and, when true, we additionally require the computed intersection x -coordinate to be strictly greater than $\alpha.x$ (i.e., the crossing lies on the ray, not behind it). We resolve (c) with an explicit $O(n)$ pre-check that tests whether α lies on any edge segment, handled *before* applying the parity rule (since “on boundary” is a separate third outcome, not meaningfully “inside” or “outside”).

2.2 Formal Pseudocode

Algorithm 2: Point-in-Polygon via Ray Casting (Even–Odd Rule)

Input: Polygon $P = (v_0, \dots, v_{n-1})$ (simple, vertices in order), query point α

Output: One of INSIDE, OUTSIDE, ON BOUNDARY

1 **Function** OnSegment(α, a, b):

 // true iff α lies on the closed segment ab

2 $cross \leftarrow (b.x - a.x)(\alpha.y - a.y) - (b.y - a.y)(\alpha.x - a.x);$

3 **if** $cross \neq 0$ **then**

4 **return** FALSE;

5 **return**

$(\min(a.x, b.x) \leq \alpha.x \leq \max(a.x, b.x))$ **and** $(\min(a.y, b.y) \leq \alpha.y \leq \max(a.y, b.y));$

6 **Function** PointInPolygon(P, α):

 // Step 1: explicit boundary check, $O(n)$

7 **for** $i \leftarrow 0$ **to** $n - 1$ **do**

8 $j \leftarrow (i + 1) \bmod n;$

9 **if** OnSegment(α, v_i, v_j) **then**

10 **return** ON BOUNDARY;

 // Step 2: ray-casting parity count, $O(n)$

11 $count \leftarrow 0;$

12 **for** $i \leftarrow 0$ **to** $n - 1$ **do**

13 $j \leftarrow (i + 1) \bmod n;$

14 **if** $(v_i.y > \alpha.y) \neq (v_j.y > \alpha.y)$ **then**

15 $x_{cross} \leftarrow v_i.x + \frac{\alpha.y - v_i.y}{v_j.y - v_i.y} (v_j.x - v_i.x);$

16 **if** $\alpha.x < x_{cross}$ **then**

17 $count \leftarrow count + 1;$

18 **if** $count$ is odd **then**

19 **return** INSIDE;

20 **else**

21 **return** OUTSIDE;

2.3 Complexity Analysis

Time: Step 1 (boundary check) iterates over all n edges, doing $O(1)$ arithmetic work (one cross product and two interval containment tests) per edge: $O(n)$ total. Step 2 (parity count) again iterates over all n edges once, doing $O(1)$ work per edge (two comparisons, one division, one comparison): $O(n)$ total. Hence the overall running time is

$$O(n) + O(n) = O(n).$$

Space: Besides the $O(n)$ space to store the input polygon, the algorithm uses only $O(1)$ auxiliary space (a counter and a few temporary variables), so auxiliary space is $O(1)$ and total space is $O(n)$.

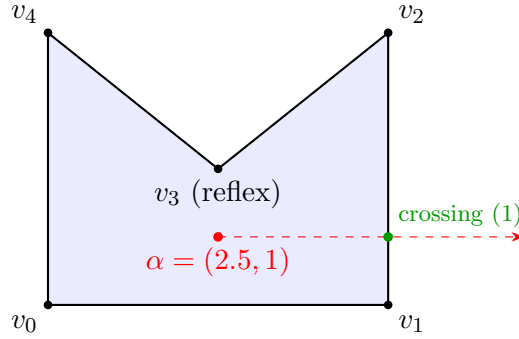


Figure 1: Ray casting example. The horizontal ray from $\alpha = (2.5, 1)$ crosses the polygon boundary exactly once (edge v_1v_2 at $x = 5$); it does not cross the top notch since that notch lies entirely above $y = 2$. An odd crossing count (1) correctly reports α as INSIDE. In contrast, the point $(2.5, 3)$, which lies inside the downward notch, yields an even crossing count (2) and is correctly reported OUTSIDE, illustrating that the algorithm correctly handles non-convex polygons.

2.4 Proof of Correctness

Theorem 2 (Jordan Parity Theorem for Simple Polygons). *Let P be a simple polygon and let $\alpha \notin \partial P$ be a point such that the horizontal ray R from α in the $+x$ direction meets ∂P only at points that are not tangential touches occurring at more than the vertex incidences handled below. Then α is interior to P if and only if R crosses ∂P an odd number of times.*

Proof. Consider moving a point along R starting from $x = +\infty$ toward α . Since P is bounded, the point starts strictly outside P . The Jordan Curve Theorem guarantees that the simple closed curve ∂P separates the plane into exactly two connected regions (interior and exterior); therefore, every time the moving point crosses ∂P transversally, it switches from one region to the other (it cannot stay in the same region across a proper crossing, and it cannot jump directly between the two regions without crossing the separating curve). Hence, after k crossings have been passed, the point's region membership is interior iff k is odd (starting parity $0 = \text{exterior}$). Setting the moving point's position to α itself, the number of crossings passed is exactly the number of crossings of R with ∂P between α and infinity — precisely what our algorithm counts. Hence α is interior iff this count is odd. ■

Claim 1 (Correct handling of vertex incidences). *Suppose the ray through α passes exactly through a vertex v_i (i.e., $v_i.y = \alpha.y$), with neighbors v_{i-1}, v_{i+1} . The strict-inequality rule contributes an even total count from the two edges incident to v_i when the boundary merely touches the ray at v_i without crossing it (both neighbors on the same side), and an odd total count (exactly 1) when the boundary properly passes through v_i from one side to the other (neighbors on opposite sides). Hence parity is preserved correctly in all cases.*

Proof. Since $v_i.y = \alpha.y$, the comparison $(v_i.y > \alpha.y)$ evaluates to FALSE. Consider the two edges (v_{i-1}, v_i) and (v_i, v_{i+1}) .

Case 1 (touch, not cross): both $v_{i-1}.y > \alpha.y$ and $v_{i+1}.y > \alpha.y$. Edge (v_{i-1}, v_i) : (TRUE) $\neq$ (FALSE) holds $\Rightarrow$ counted. Edge (v_i, v_{i+1}) : (FALSE) $\neq$ (TRUE) holds $\Rightarrow$ counted. Total contribution from these two edges: 2, which is even — correctly reflecting that the boundary dips down to touch the ray at v_i and returns to the same side, i.e., no true crossing occurs. The symmetric case (both neighbors below) analogously gives: edge (v_{i-1}, v_i) : (FALSE) $\neq$ (FALSE) does not hold $\Rightarrow$ not

counted; edge (v_i, v_{i+1}) : $(\text{FALSE}) \neq (\text{FALSE})$ does not hold $\Rightarrow$ not counted. Total contribution: 0, again even.

Case 2 (proper crossing through vertex): $v_{i-1}.y > \alpha.y$ and $v_{i+1}.y < \alpha.y$ (or symmetrically). Edge (v_{i-1}, v_i) : $(\text{TRUE}) \neq (\text{FALSE})$ holds $\Rightarrow$ counted. Edge (v_i, v_{i+1}) : $(\text{FALSE}) \neq (\text{FALSE})$, since $v_{i+1}.y < \alpha.y$ is not $> \alpha.y$ does not hold $\Rightarrow$ not counted. Total contribution: 1, odd — correctly reflecting a single genuine crossing of the boundary through v_i .

In both cases the local contribution has the correct parity (even for a touch, odd for a crossing), so the overall count's parity — and hence the algorithm's INSIDE/OUTSIDE decision — is unaffected by vertex incidences. ■

Claim 2 (Correct handling of horizontal edges). *An edge (v_i, v_j) with $v_i.y = v_j.y$ (a horizontal edge not coincident with α on the boundary — that case is caught by the Step 1 pre-check) never satisfies the crossing condition, and hence contributes 0 to the count.*

Proof. If $v_i.y = v_j.y$, then $(v_i.y > \alpha.y)$ and $(v_j.y > \alpha.y)$ are evaluated on identical operands and thus are always equal (both TRUE or both FALSE), so the condition $(v_i.y > \alpha.y) \neq (v_j.y > \alpha.y)$ is always FALSE. This is correct: a horizontal edge either lies entirely above, entirely below, or exactly on the line $y = \alpha.y$; in the first two cases it clearly cannot cross a ray traveling along $y = \alpha.y$, and the third case (collinear with the ray) is precisely handled separately by the Step 1 boundary check, since if $\alpha.y$ equals the edge's y -coordinate and $\alpha.x$ falls within the edge's x -range, α lies on that edge and is reported ON BOUNDARY before the ray test ever runs. ■

Theorem 3 (Correctness of Algorithm 2). *Algorithm 2 correctly outputs ON BOUNDARY if $\alpha \in \partial P$, and otherwise correctly outputs INSIDE or OUTSIDE according to whether α lies in the interior or exterior of P .*

Proof. If α lies on some edge of P , Step 1 detects this exactly (the cross-product test certifies collinearity with the edge's supporting line, and the bounding-box test certifies that α lies between the endpoints, together exactly characterizing membership in the closed segment), and the algorithm correctly returns ON BOUNDARY without proceeding further.

Otherwise, $\alpha \notin \partial P$, and by Theorem 2, α is interior to P iff the ray from α crosses ∂P an odd number of times. The two preceding claims establish that the algorithm's count (using the strict-inequality rule together with the “crossing is strictly to the right of α ” check, which correctly restricts attention to crossings that lie on the forward ray rather than behind α) has exactly the same parity as the true geometric crossing count of R with ∂P , for every edge configuration (ordinary edges, edges touching the ray only at a vertex, and horizontal edges). Since the algorithm's output (INSIDE iff *count* is odd) matches the true parity, correctness follows. ■

3 Problem 3: Minimizing Weighted Sum of Completion Times (10 points)

Problem Statement

There are n jobs to be scheduled on a single machine; job i has processing time t_i and weight w_i . Given an ordering (schedule) of the jobs, let C_i denote the completion time of job i under that ordering. Design an efficient algorithm to find an ordering of the jobs minimizing $\sum_{i=1}^n w_i C_i$.

3.1 Algorithm Idea & Intuition

Intuitively, a job should be scheduled earlier if it is either *quick* (small t_i , so it does not delay everyone behind it much) or *important* (large w_i , so its own delay is costly). This suggests ranking jobs by the ratio

$$r_i = \frac{w_i}{t_i},$$

and scheduling them in **decreasing order of r_i** — this is known as the **Weighted Shortest Processing Time (WSPT) rule**, or **Smith’s Rule**. We will prove this greedy ordering is optimal via a classical **exchange argument**: we show that any schedule containing an “inversion” (a pair of adjacent jobs violating the decreasing-ratio order) can be improved (or at least not worsened) by swapping that pair, and hence any schedule that cannot be improved by such a swap must already be sorted by decreasing ratio.

3.2 Formal Pseudocode

Algorithm 3: WSPT (Smith’s Rule) Scheduling

Input: n jobs with processing times $t_1, \dots, t_n > 0$ and weights $w_1, \dots, w_n > 0$

Output: An ordering $\sigma = (\sigma_1, \dots, \sigma_n)$ minimizing $\sum_{i=1}^n w_i C_i$, and the optimal objective value

```

1 for  $i \leftarrow 1$  to  $n$  do
2    $r_i \leftarrow w_i/t_i$ ;
3 Sort jobs into the permutation  $\sigma = (\sigma_1, \dots, \sigma_n)$  such that  $r_{\sigma_1} \geq r_{\sigma_2} \geq \dots \geq r_{\sigma_n}$ ;
   //  $O(n \log n)$ , e.g. mergesort
4  $C \leftarrow 0$ ;
5  $TotalCost \leftarrow 0$ ;
6 for  $k \leftarrow 1$  to  $n$  do
7    $C \leftarrow C + t_{\sigma_k}$ ;
8    $C_{\sigma_k} \leftarrow C$ ; // completion time of job  $\sigma_k$ 
9    $TotalCost \leftarrow TotalCost + w_{\sigma_k} \cdot C$ ;
10 return  $(\sigma, TotalCost)$ ;
```

3.3 Complexity Analysis

Time: Computing all n ratios takes $O(n)$. Sorting the n jobs by ratio takes $O(n \log n)$ (using any comparison sort, e.g., mergesort, per the course’s stated assumption that sorting costs $O(n \log n)$). The final loop computing completion times and accumulating the weighted sum performs $O(1)$

work per job, i.e., $O(n)$ total. Hence the overall running time is

$$O(n) + O(n \log n) + O(n) = O(n \log n),$$

dominated by the sorting step. **Space:** $O(n)$ to store the ratios, the permutation σ , and completion times.

3.4 Proof of Correctness (Exchange Argument)

Lemma 3 (Adjacent Exchange Lemma). *Let S be any schedule in which job i is scheduled immediately before job j (i.e., j comes right after i , with no jobs in between), and let S' be the schedule identical to S except that i and j are swapped (so j now immediately precedes i). Let C be the completion time of the last job scheduled before this pair in S (equivalently, in S' ; this quantity is common to both since all other jobs are unaffected). Then*

$$\text{cost}(S) - \text{cost}(S') = w_j t_i - w_i t_j,$$

where $\text{cost}(\cdot)$ denotes the total weighted completion time of the schedule. In particular, if $\frac{w_i}{t_i} < \frac{w_j}{t_j}$, then $\text{cost}(S) > \text{cost}(S')$, i.e., swapping i and j strictly decreases the total cost.

Proof. Only the completion times (and hence weighted contributions) of i and j change between S and S' ; every other job's completion time is unaffected, since the pair $\{i, j\}$ occupies the same two consecutive time slots in both schedules, and jobs scheduled after this pair see the same combined block of processing time $(t_i + t_j)$ elapse before them regardless of the internal order of i, j .

In S (order i then j): $C_i = C + t_i$, and $C_j = C + t_i + t_j$. The combined contribution of i, j to the objective is

$$w_i(C + t_i) + w_j(C + t_i + t_j) = (w_i + w_j)C + w_i t_i + w_j t_i + w_j t_j.$$

In S' (order j then i): $C'_j = C + t_j$, and $C'_i = C + t_j + t_i$. The combined contribution is

$$w_j(C + t_j) + w_i(C + t_j + t_i) = (w_i + w_j)C + w_j t_j + w_i t_j + w_i t_i.$$

Subtracting,

$$\text{cost}(S) - \text{cost}(S') = [(w_i + w_j)C + w_i t_i + w_j t_i + w_j t_j] - [(w_i + w_j)C + w_j t_j + w_i t_j + w_i t_i] = w_j t_i - w_i t_j.$$

Now, $\frac{w_i}{t_i} < \frac{w_j}{t_j} \iff w_i t_j < w_j t_i$ (multiplying both sides by the positive quantity $t_i t_j$) $\iff w_j t_i - w_i t_j > 0$. Hence $\text{cost}(S) - \text{cost}(S') > 0$, i.e., $\text{cost}(S') < \text{cost}(S)$. ■ □

Theorem 4 (Global Optimality of the WSPT Order). *The schedule σ produced by Algorithm 3 (sorted in decreasing order of w_i/t_i) minimizes $\sum_i w_i C_i$ over all possible orderings of the n jobs.*

Proof. Let S^* be any optimal schedule. We claim S^* must already have no adjacent pair of jobs (i, j) (with i immediately before j) satisfying $w_i/t_i < w_j/t_j$ — i.e., S^* has no “inversion” with respect to the ratio order.

Suppose, for contradiction, that S^* contains such an adjacent inversion (i, j) . By Lemma 3, swapping i and j produces a schedule S' with $\text{cost}(S') < \text{cost}(S^*)$, contradicting the optimality of S^* . Hence no such inversion can exist in S^* : for every pair of adjacent jobs in S^* , the ratios must be in non-increasing order, i.e., S^* is sorted by decreasing w_i/t_i (ties broken arbitrarily, which by

Lemma 3 incur $\text{cost}(S) - \text{cost}(S') = w_j t_i - w_i t_j = 0$ when ratios are equal, so tie-breaking order does not affect the cost).

Since *any* two total orderings that are both sorted by non-increasing w_i/t_i differ from each other only in how they break ties among equal-ratio jobs, and such tie-breaking swaps leave the cost unchanged (by the $w_j t_i - w_i t_j = 0$ case of Lemma 3), every ratio-sorted ordering — including the one σ produced by Algorithm 3 — achieves the same cost as S^* . Hence

$$\text{cost}(\sigma) = \text{cost}(S^*) = \min_{\text{orderings } S} \text{cost}(S),$$

i.e., σ is optimal. ■

□

Corollary 1. *Algorithm 3 correctly computes an optimal schedule and its optimal cost in $O(n \log n)$ time.*

4 Problem 4: Cheapest Reachable Node in a DAG (10 points)

Problem Statement

Given a directed acyclic graph $G = (V, E)$ where every node $u \in V$ has a positive integer $\text{price}(u)$, define

$$\text{cost}(u) = \min_{v \text{ reachable from } u \text{ (including } u \text{ itself)}} \text{price}(v).$$

Design an algorithm that computes $\text{cost}(u)$ for every $u \in V$.

4.1 Algorithm Idea & Intuition

The recursive structure of reachability in a DAG gives an immediate dynamic-programming recurrence:

$$\text{cost}(u) = \min \left(\text{price}(u), \min_{(u,v) \in E} \text{cost}(v) \right),$$

because every node reachable from u is either u itself, or reachable via some first edge (u, v) followed by a path from v — i.e., the set of nodes reachable from u (other than u) is exactly the union, over all out-neighbors v , of the nodes reachable from v .

For this recurrence to be evaluated directly (bottom-up, without recursion/memoization overhead), we need to process nodes in an order such that *all out-neighbors of u are processed before u itself*. Since G is acyclic, such an order always exists: it is exactly the **reverse of a topological order** of G (equivalently: process sinks first, sources last). We obtain this order for free as a by-product of a single DFS-based topological sort: if L is the list of vertices in the order they *finish* during DFS (i.e., in the order `DFS-Visit` returns), then L is already in reverse topological order, because for every edge (u, v) , v is guaranteed to finish before u in DFS on a DAG (proved below). We can therefore process L directly, left to right, without needing to reverse or pop anything from a stack.

4.2 Formal Pseudocode

Algorithm 4: Minimum Reachable Price via DFS-based Reverse-Topological DP

Input: DAG $G = (V, E)$ given by adjacency lists, and $\text{price}(u)$ for each $u \in V$

Output: $\text{cost}(u)$ for every $u \in V$

```

1 Function DFS-Visit( $u, L$ ):
2    $\text{visited}[u] \leftarrow \text{TRUE};$ 
3   foreach  $v$  such that  $(u, v) \in E$  do
4     if not  $\text{visited}[v]$  then
5       DFS-Visit( $v, L$ );
6   append  $u$  to  $L$ ;                                     //  $u$  finishes now

7 Function FinishOrder( $G$ ):
8   foreach  $u \in V$  do
9      $\text{visited}[u] \leftarrow \text{FALSE};$ 
10   $L \leftarrow$  empty list;
11  foreach  $u \in V$  do
12    if not  $\text{visited}[u]$  then
13      DFS-Visit( $u, L$ );
14  return  $L$ ;                                           //  $L$  is in increasing order of finish time = reverse
                                                         topological order

15 Function ComputeCost( $G, \text{price}(\cdot)$ ):
16   $L \leftarrow \text{FinishOrder}(G);$ 
17  foreach  $u \in L$  (in the order they appear in  $L$ ) do
18     $\text{cost}[u] \leftarrow \text{price}(u);$ 
19    foreach  $v$  such that  $(u, v) \in E$  do
20      if  $\text{cost}[v] < \text{cost}[u]$  then
21         $\text{cost}[u] \leftarrow \text{cost}[v];$ 
22  return  $\text{cost}[\cdot];$ 

```

4.3 Complexity Analysis

Time: $\text{FinishOrder}()$ is a standard DFS: every vertex is visited once and every edge is examined exactly once across all calls to $\text{DFS-Visit}()$, giving $O(|V| + |E|)$. The subsequent loop in $\text{ComputeCost}()$ iterates over each $u \in L$ exactly once ($O(1)$ work for the initialization $\text{cost}[u] \leftarrow \text{price}(u)$), and for each u examines its out-edges; summed over all u , the total number of edge examinations is $\sum_u \deg^+(u) = |E|$. Hence the DP pass costs $O(|V| + |E|)$ as well. The overall running time is

$$O(|V| + |E|) + O(|V| + |E|) = O(|V| + |E|),$$

which is linear in the size of the graph, as required. **Space:** $O(|V| + |E|)$ for the adjacency-list representation, plus $O(|V|)$ for the visited , cost arrays and the list L , plus $O(|V|)$ recursion-stack depth in the worst case (a long directed path) — overall $O(|V| + |E|)$.

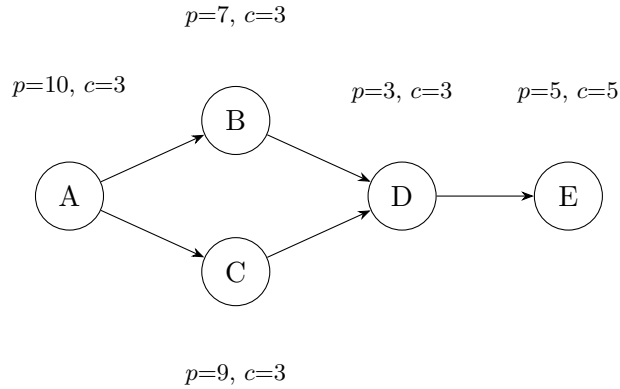


Figure 2: Example DAG with $\text{price}(\cdot) = p$ shown alongside the computed $\text{cost}(\cdot) = c$. Processing in reverse topological order: E (sink) first, $\text{cost}(E) = 5$; then D : $\text{cost}(D) = \min(3, \text{cost}(E)) = \min(3, 5) = 3$; then B : $\text{cost}(B) = \min(7, \text{cost}(D)) = \min(7, 3) = 3$; then C : $\text{cost}(C) = \min(9, \text{cost}(D)) = \min(9, 3) = 3$; finally A : $\text{cost}(A) = \min(10, \text{cost}(B), \text{cost}(C)) = \min(10, 3, 3) = 3$.

4.4 Proof of Correctness

Lemma 4 (Finish-Time Ordering on a DAG). *In any DFS traversal of a DAG $G = (V, E)$, for every edge $(u, v) \in E$, v finishes strictly before u .*

Proof. Consider the moment the edge (u, v) is examined during $\text{DFS-VISIT}(u, \cdot)$ (i.e., u has been discovered but not yet finished). There are two cases.

- v is undiscovered at this moment: then the algorithm immediately calls $\text{DFS-VISIT}(v, \cdot)$, making v a descendant of u in the DFS forest. Since $\text{DFS-VISIT}(u, \cdot)$ does not append u to L (i.e., does not finish) until *all* recursive calls it makes have returned — including the call on v , which only returns after v has been appended to L — we get v finishes strictly before u .
- v is already discovered at this moment: then v cannot be an ancestor of u currently on the recursion stack, for if it were, the edge (u, v) would close a cycle $v \rightarrow \dots \rightarrow u \rightarrow v$, contradicting that G is acyclic. Hence v is not an active ancestor, so v must have already *finished* (its entire $\text{DFS-VISIT}()$ call, including appending it to L , has already completed) before u 's call to examine edge (u, v) — in particular, before u itself finishes.

In both cases, v finishes strictly before u . ■

□

Corollary 2. *The list L returned by $\text{FinishOrder}()$ — vertices in increasing order of finish time — has the property that for every edge $(u, v) \in E$, v appears strictly before u in L . Equivalently, when $\text{ComputeCost}()$ processes L left to right, every out-neighbor v of u is processed (and has $\text{cost}[v]$ finalized) before u is processed.*

Theorem 5 (Correctness of Algorithm 4). *For every $u \in V$, the value $\text{cost}[u]$ computed by $\text{ComputeCost}()$ equals $\text{cost}(u) = \min_{w \text{ reachable from } u} \text{price}(w)$.*

Proof. We proceed by strong induction on the processing order given by L (equivalently, induction on finish time).

Base case. Let u be the first vertex processed (smallest finish time in L). By Lemma 4, if u had any outgoing edge (u, v) , then v would have an even smaller finish time than u , contradicting that u is first. Hence u has no outgoing edges, so the only node reachable from u is u itself, and $\text{cost}(u) = \text{price}(u)$. The algorithm sets $\text{cost}[u] \leftarrow \text{price}(u)$ and (having no out-edges) never updates it further — correct.

Inductive step. Let u be processed at some later point, and assume (induction hypothesis) that $\text{cost}[v] = \text{cost}(v)$ has already been correctly computed for every vertex v processed strictly before u in L . By Corollary 2, this includes every out-neighbor v with $(u, v) \in E$.

The set of vertices reachable from u decomposes as

$$\text{Reach}(u) = \{u\} \cup \bigcup_{(u,v) \in E} \text{Reach}(v),$$

since any node $w \neq u$ reachable from u is reached by a path that departs u via some first edge (u, v) and then continues as a path from v to w (i.e., $w \in \text{Reach}(v)$); conversely every node in $\text{Reach}(v)$ for an out-neighbor v is clearly reachable from u . Taking $\min \text{price}(\cdot)$ over both sides,

$$\text{cost}(u) = \min_{w \in \text{Reach}(u)} \text{price}(w) = \min \left(\text{price}(u), \min_{(u,v) \in E} \min_{w \in \text{Reach}(v)} \text{price}(w) \right) = \min \left(\text{price}(u), \min_{(u,v) \in E} \text{cost}(v) \right).$$

By the induction hypothesis, $\text{cost}(v) = \text{cost}[v]$ (the algorithm's already-computed value) for every out-neighbor v of u . Hence

$$\text{cost}(u) = \min \left(\text{price}(u), \min_{(u,v) \in E} \text{cost}[v] \right),$$

which is exactly the value the algorithm computes for $\text{cost}[u]$ (initializing to $\text{price}(u)$, then taking the minimum with $\text{cost}[v]$ over each out-edge (u, v) in turn). Thus $\text{cost}[u] = \text{cost}(u)$, completing the induction.

By induction over all $|V|$ positions in L , $\text{cost}[u] = \text{cost}(u)$ for every $u \in V$. ■

□